ChatGPT

Reproducing PHP2Shell Heap-Leak (Stage 1) and Local Lab Setup

Executive Summary: The **php2shell** flaws (CVE-2026-109618/109619) allow **pre-auth remote code execution (RCE)** in PHP by exploiting out-of-memory (OOM) and stale-state bugs in the **session.upload_progress** path【37†L133-L142】【25†L245-L254】. To reproduce the *heap-leak* stage safely, you need an exact vulnerable PHP build (e.g. PHP 8.2.x < 8.2.35) on Apache with **mod_php** (prefork MPM), one worker, and specific PHP settings (see below)【18†L307-L314】【41†L176-L184】. In the lab, a crafted multipart upload triggers an OOM before PHP finishes initializing session state. The session's `PS(id)` pointer remains stale, so a *subsequent* request on the same process leaks heap contents in a `PHPSESSID` cookie【25†L245-L254】【25†L252-L259】. Using the devploit/php2shell toolkit with pre-recorded **offset profiles** for your PHP build, you can detect the leak and (with calibration) proceed to later RCE stages. This report details prerequisites, configuration, the leak mechanism, common failure modes, and step-by-step commands to run the toolkit on `http://localhost:8080` in a controlled environment. We also compare required vs default settings and include a checklist for a reproducible lab.

Prerequisites

Component	Required (Lab)	Typical Default
PHP version	Vulnerable (e.g. 8.2.12) (patched in 8.2.35) 【37†L133-L142】	Any (may include patched or EOL versions)
PHP SAPI	Apache **mod_php** (prefork)【18†L307-L314】	mod_php or FPM
Architecture	Match your target (x86_64 or arm64)【18†L318-L322】【19†L316-L324】	Platform default
Workers (prefork)	**1** (single Apache child)【18†L307-L314】【19†L262-L270】	Multiple (default >1)
Session handler	`session.serialize_handler = php` (default)	`php`
Session upload	Enabled `session.upload_progress.enabled=On` (needed)【18†L307-L314】【41†L191-L196】	Disabled (default Off)
Display errors	On (`display_errors=On`) (lab mode)【18†L307-L314】【32†L592-L600】	Off (production default)
memory_limit	Low (e.g. `8M`) to force OOM【41†L176-L184】	`128M` (default)
expose_php	Off (recommended)【27†L66-L72】	On (usually On)

Component	Required (Lab)	Typical Default
session.use_strict_mode	0 (default)	Usually Off
session.use_cookies	1 (default)	1 (default)

- **Exact PHP build:** The exploit chain requires the exact PHP build (version + OS + architecture). Offsets/profiles differ between builds【27†L66-L72】【28†L209-L219】. For example, devploit's toolkit has verified profiles for PHP 8.4.26 (amd64) and 8.5.11 (aarch64/amd64) but *not* for 8.2.12. You may need to generate a profile for 8.2.12 (see *Calibration* below) or test on a known vulnerable build.
- **Server API:** PHP must run as Apache's prefork mod_php (not FPM or CGI). The public exploit operates in the Apache worker before any user code runs【9†L19-L24】【32†L608-L614】. (PHP-FPM is not supported by the shipped chain.)
- **One worker:** Only one Apache child should be handling requests (or use devploit's **pin-workers** to isolate a single process)【18†L323-L330】【32†L608-L614】. Otherwise the OOM-induced stale pointer won't persist across requests on the same process.
- **Module enabled:** The attack relies on PHP's *session upload-progress* feature, which is **disabled by default**. Ensure `session.upload_progress.enabled=On` in php.ini (or use `php_admin_value` in Apache conf)【41†L191-L196】.

If any of these prerequisites are not met (e.g. patched PHP, FPM, multiple workers, upload progress off), the heap-leak stage will **not trigger**. For example, devploit notes that PHP 8.2 has the stale state but **no cookie leak** by default【19†L320-L324】.

Required Configuration vs Defaults

For a reproducible lab, configure PHP/Apache to:

- `session.upload_progress.enabled = On` – Enables the pre-RINIT upload callback【18†L307-L314】【41†L191-L196】 (default is *Off*).
- `display_errors = On` – Allows error messages (including the memory_limit error) to appear in output【32†L592-L600】 (default *Off* in prod).
- `memory_limit = 8M` – Low limit forces an out-of-memory condition during request parsing【41†L176-L184】 (default 128M).
- `expose_php = Off` – Not strictly required, but omitting the PHP header makes memory layout more stable【27†L66-L72】.
- **Apache MPM (prefork):** Limit to 1 worker. For example, in `apache2.conf` set `ServerLimit 1`, `StartServers 1`, `MaxRequestWorkers 1`. This ensures a single child holds the stale state【32†L608-L614】 (default allows many workers).
- **Bind to localhost:** Restrict the webserver to `127.0.0.1` (or Docker's loopback) so the exploit can only be tested locally. For example, run Docker with `-p 127.0.0.1:8080:80`.
- **Use HTTP (not HTTPS):** The devploit chain is verified over plain HTTP. (HTTPS with SNI is experimental and may fail the exploit【19†L325-L332】.)

The table below compares these requirements to typical defaults:

Setting	Required for Lab	Typical Default
`session.upload_progress`	**On** (Enabled)【41†L191-L196】	Off (disabled)
`display_errors`	**On**【32†L592-L600】	Off (disabled)
`memory_limit`	*e.g.* `8M`【41†L176-L184】	128M
`expose_php`	**Off** (recommended)【27†L66-L72】	On
Apache worker model	**1 child**【32†L608-L614】	Multiple (prefork)
PHP SAPI	**Apache mod_php**	mod_php or FPM

Checklist: Before running the exploit, verify:

- PHP version and architecture (`php -v` , `uname -a` or `phpinfo()`).
- Apache's SAPI (`apachectl -M` should show `mpm_prefork_module` and `mod_php`).
- PHP settings (`phpinfo()` or `php -i`) for `session.upload_progress.enabled` , `display_errors` , `memory_limit` .
- If in Docker: a custom php.ini mounted and Apache conf modified for 1 worker.
- Bound to localhost only (`netstat` or `docker ps` showing 127.0.0.1:8080).

Heap-Leak Mechanism (Stage 1)

In brief, the attacker sends a **multipart/form-data POST** with many fields to exhaust PHP's memory. PHP initializes session state early (in the upload-progress callback), setting the global `PS(id)` (PHPSESSID) pointer. If an OOM **abort** occurs *before* PHP can clean up, the allocator frees the request's heap but **leaves** `PS(id)` **pointing to the freed data**【25†L245-L254】. On the **next** multipart request to the *same worker*, PHP still sees that stale `PS(id)` as a valid `zend_string` . When it URL-encodes this string for the `Set-Cookie: PHPSESSID=…` header, it instead dumps contents of the old (freed) heap. This leaks attacker-controlled bytes from the server's memory back to the client【25†L252-L259】.

```
flowchart LR
    A[Send large multipart upload (session.start via upload_progress)] --> B[OOM
occurs; PHP aborts request<br>but leaves PS(id) pointer stale]
    B --> C[New multipart request on same worker]
    C --> D[PHP reaches upload-progress cleanup,<br>uses stale PS(id) as
zend_string]
    D --> E[Stale heap contents URL-encoded in PHPSESSID cookie (memory leak)]
```

For example, the response might include a header like:

```
Set-Cookie: PHPSESSID=a%3A1%3A%7Bi%3A0%3Bs%3A12%3A%22leaked_data%22%3B%7D;
path=/; HttpOnly
```

where the `%xx` data encodes heap bytes from the previous request【25†L252-L259】. (Note: actual payloads are JSON-like or binary in practice, depending on the PHP serializer.) The key point is that this *cookie value reveals memory addresses*, including the chunk base (heap) and `libphp` base【25†L252-L259】. Once leaked, the exploit uses these addresses (offsets) to craft the later RCE stages.

Offsets and Profiles

Because of ASLR and build differences, **every PHP build has different offsets** for internal functions (e.g. `zif_system`, `zval_ptr_dtor`) and heap layouts【27†L66-L72】【28†L209-L219】. The devploit toolkit uses **pre-calculated "profiles"** of offsets and heap parameters for specific images【28†L209-L219】. A profile ties a PHP version/OS/arch to:

- Symbol offsets in `libphp.so` (e.g. the address of `zif_system`)【28†L209-L219】.
- Heap-chunk and table indices needed to overwrite a fake structure in stage 3 (RCE).
- Verified flags (`heap leak: yes/no`, `RCE: yes/no`)【18†L307-L314】.

If your target matches a known profile, the dispatcher will use those offsets explicitly【18†L318-L322】. Otherwise it aborts (to avoid blind guessing). You can force a profile with `--profile`. For example:

```
python3 tools/php2shell_auto.py -u http://localhost:8080/ -–profile php-8.4.26-bookworm-amd64
```

would run the exploit chain using the 8.4.26-amd64 offsets (if your server is indeed PHP 8.4.26 on Debian Bookworm). Without matching offsets, stage 2/3 will fail. In summary: **offsets matter** because the exploit must know exactly where to write and jump in memory. In a lab, you either use a build that already has offsets, or use `php2shell_calibrate.py` to generate new profiles (see below).

Common Failure Modes and Diagnostics

The heap-leak may **silently fail** if conditions aren't met. Some common reasons:

- **Patch Level:** If PHP ≥ 8.2.35 (or 8.4.27, 8.5.12, etc.) on the target, the bug is fixed【37†L133-L142】. The tool will detect a "fixed" banner and stop.
- **Upload progress disabled:** No callback → no stale pointer, no leak【32†L592-L600】【41†L191-L198】.
- **Display errors off:** The exploit relies on seeing the OOM error message. With `display_errors=Off`, stage 1 (cookie leak) **does not trigger** in the shipped payload【32†L592-L600】. (Research has alternative methods for that case, but devploit's profiles assume it's On.)
- **Multiple workers:** Without pinning, the chain's first and second requests may land in *different* processes. This breaks the stale-state exploit. Use `--pin-workers N` (N = total workers) or configure Apache to 1 worker to ensure the same process handles both requests【18†L323-L330】【32†L608-L614】.
- **Non-multipart endpoint:** The exploit only works via multipart POST (RFC1867). Ordinary requests to a normal PHP page without file-upload processing won't reach the vulnerable code path. Ensure the target URL handles form data (e.g. a simple PHP file can work).

- **HTTPS/SNI issues:** The chain's HTTPS support is experimental【19†L325-L332】. For now, use plain HTTP in lab.
- **Wrong URL format:** The tool expects a URL to a PHP endpoint. For example, if testing via devploit's labs, use `-u http://127.0.0.1:8105/index.php`, not just the domain. Check `php2shell_auto --dry-run` if unsure which URL to use【19†L283-L292】.

If the exploit appears to hang or fail quietly, use diagnostics:

- **Memory-limit probe:** The tool by default sends a test request to probe `memory_limit` (it varies with PHP config)【32†L478-L487】. Check the output for a line like `OK detected memory_limit=8M`.
- **Dry-run mode:** `php2shell_auto.py --dry-run` will detect the PHP version and tell you what it would do without attacking【18†L352-L359】.
- **Assess mode:** The `php2shell_assess.py` script (or `--dry-run --no-color`) can attempt just the leak/disclosure without proceeding to RCE【32†L478-L487】【32†L589-L598】. It will report if a heap leak was found.
- **Apache logs:** Look for repeated `PHP Fatal error: Allowed memory size of ... bytes exhausted` messages, or any `zend_mm_heap corrupted` logs – signs of OOM exploitation attempts【32†L599-L604】【25†L233-L239】.
- **Response headers:** After the first OOM request, check if a `Set-Cookie: PHPSESSID` appears with a long or hex-encoded value. That is direct proof of a leak【25†L252-L259】.
- **Worker restarts:** Unexplained worker crashes or restarts during testing can indicate the exploit triggered a fatal condition.

In devploit's example run (PHP 8.4.26 lab), the output shows detection and a final "PRE-AUTH RCE CONFIRMED" with proof:

```
00:00.16  OK         detected memory_limit=8M
   │ [+] chunk base = 0x7ffffc400000
   │ [+] libphp base = 0x7ffffe013000
   │ PROOF: uid=33(www-data) gid=33(www-data)
PRE-AUTH RCE CONFIRMED
```

【18†L370-L374】. The `chunk base` and `libphp base` lines come from the heap and library leaks.

Safe Local Lab Setup

Using devploit's toolkit (recommended): Clone the repo and use its lab manager:

```
git clone https://github.com/devploit/php2shell.git
cd php2shell
python3 tools/bootstrap_upstream.py
python3 tools/php2shell_labs.py up        # Start labs (Docker)
python3 tools/php2shell_labs.py status    # Verify labs are running
```

By default, this brings up several PHP-Apache containers (e.g. 7.4, 8.0, ..., 8.5) bound to localhost on ports 8100–8106【19†L252-L261】. For example, PHP 8.4.26 runs at `127.0.0.1:8105`. To add PHP 8.2 (which devploit doesn't predefine), you could manually run a container:

```
docker run -d --name p2s-lab-8212 \
  -v /path/to/lab.ini:/usr/local/etc/php/conf.d/lab.ini \
  -v /path/to/apache.conf:/etc/apache2/conf-enabled/mpm.conf \
  -p 127.0.0.1:8080:80 \
  php:8.2.12-apache
```

Where `lab.ini` sets the above PHP flags (e.g. `session.upload_progress.enabled=On`, `display_errors=On`, `memory_limit=8M`) and `apache.conf` forces one process. The container listens only on localhost:8080.

Manual Docker example: For simplicity, you could use a single container:

```
# Create a custom php.ini snippet for the lab:
cat > lab.ini <<EOF
session.upload_progress.enabled = On
session.use_strict_mode = 0
display_errors = On
memory_limit = 8M
expose_php = Off
EOF

# Run PHP 8.2.12 container bound to localhost
docker run --rm -d --name php2shell-lab \
  -v $(pwd)/lab.ini:/usr/local/etc/php/conf.d/lab.ini \
  -p 127.0.0.1:8080:80 \
  php:8.2.12-apache
```

Adjust Apache's MPM settings inside the container if needed (prefork with 1 worker). Always use `-p 127.0.0.1:PORT:80` to restrict to localhost.

Checklist for Lab:

- [ ] Docker or VM ready with PHP 8.2.12 and Apache (mod_php).
- [ ] PHP configs set as above (`session.upload_progress=On`, etc).
- [ ] Bound to 127.0.0.1: `docker run -p 127.0.0.1:8080:80 ...`.
- [ ] Confirm settings: `curl http://localhost:8080/phpinfo.php` (or `php -i`) shows the values.

Running the devploit Toolkit

With the lab up, use devploit's scripts:

1. **php2shell_auto.py (dispatcher):** Auto-detects PHP version/arch and runs the chain. Examples:

```
# On the devploit lab for PHP 8.4.26 (passwd and proof, version auto-detect):
python3 tools/php2shell_auto.py \
  -u http://127.0.0.1:8105/index.php \
  --container p2s-lab-8426 \
  --repo-dir vendor/php2shell-upstream
```

Expected output:

```
DISCOVER   http://127.0.0.1:8105/index.php
OK         PHP 8.4.26 · Apache/2.4.69
PREFLIGHT  php-8.4.26-bookworm-amd64
PROBE      memory_limit probe: 16 MiB body
OK         detected memory_limit=8M
EXECUTE    memory_limit=8M · huge_mb=12
  [+] chunk base = 0x7ffffc400000
  [+] libphp base = 0x7ffffe013000
PROOF: uid=33(www-data)...
PRE-AUTH RCE CONFIRMED
``` 【18†L370-L374】.


For **PHP 8.2.12**, devploit has no profile by default. You could try:

```bash
python3 tools/php2shell_auto.py \
  -u http://127.0.0.1:8080/index.php \
  --php-version 8.2.12 --arch amd64 \
  --memory-limit-mb 8 \
  --pin-workers 1
```

This forces version/arch. It will still likely stop if it has no matching offsets. At minimum, it will "PREFLIGHT php-8.2.12-amd64" or error. If no leak is confirmed (which is expected), it will report failure.

Useful options:

- `--php-version X.Y.Z` if the banner is hidden.
- `--arch amd64|aarch64` for remote targets (auto-detected on local).
- `--profile NAME` to force a known build profile (from `profiles/*.json`).
- `--memory-limit-mb N` to skip the probe and assume a limit.

- `--pin-workers N` (or `--pin-workers auto`) to keep all but one worker busy【18†L323-L330】.
- `--dry-run --no-color` just detects version/profile without sending exploit payload【18†L352-L359】.

- **php2shell_assess.py:** To do a *leak-only* test (no RCE). For example:

```
python3 tools/php2shell_assess.py -u http://localhost:8080/
```

This will attempt the memory-disclosure chain and report any leaked heap base or pointers. It should **not** execute the final payload. It's useful to validate stage 1 before proceeding. (Alternatively, `--dry-run` on `php2shell_auto.py` has a similar effect.)

1. **php2shell_calibrate.py:** Use this on a known vulnerable lab to generate a profile. For example, against the devploit 8.4.26 lab:

```
python3 tools/php2shell_calibrate.py \
  --target http://127.0.0.1:8105/index.php \
  --container p2s-lab-8426 \
  --repo-dir vendor/php2shell-upstream \
  --proof-file lab/rce/out/8.4.26/p1 \
  --distro bookworm
```

This will discover heap parameters and verify RCE, writing a new profile in `profiles/`. You could similarly calibrate a PHP 8.2.12 lab if you set one up, to generate its offsets.

Non-Destructive Verification

Before attempting RCE, you can run the toolkit in non-destructive modes:

- **Memory-limit probe:** The tool's first step is to probe PHP's memory_limit via a test upload. Watch for a line like `OK detected memory_limit=8M`. If this fails, your PHP may not be configured or reachable.
- `--dry-run` **or** `--assess`**:** These modes stop after leak testing (no RCE payload). They will report if stage 1 (and optionally stage 2) succeeded. Use these to confirm the heap-leak primitive works at all before actually exploiting.
- **Check responses manually:** Use `curl` or a browser to trigger the leak (without offset). For example, send a large multipart POST (the tool has patterns for ~1000 fields). The response should be a 500 error with a weird `PHPSESSID` in Set-Cookie. If you see that, the leak is happening (albeit without useful decoding).

Exploit Stages Summary

For reference, php2shell proceeds in **three stages**:

1. **Heap leak:** As described above, triggers OOM via multipart upload, then captures heap base from cookie【25†L252-L259】.
2. **Libphp leak:** A similar stale-state trick to leak the base address of `libphp.so`. (Devploit's payload does this after stage 1.) This gives the absolute addresses of key functions.
3. **Memory corruption & RCE:** Using the leaked addresses and large multipart bodies, the exploit crafts a fake Zend hashtable to hijack execution (calls `system()`). The final proof is often running `id` or writing a file.

Only stage 1 is an information leak. Stages 2–3 are complex. In a controlled lab (with offsets known), the toolkit combines them to achieve RCE. For local tests, you can specify a harmless command (like `-c 'id'`) or use `--interactive` mode to run commands once RCE is confirmed.

Detection and Mitigation

While running a lab, also note the detection signals and mitigations:

- **Detection (Defender):** Watch for many multipart fields + OOM errors. Unusual 500 responses with large `PHPSESSID` cookies are a red flag【25†L233-L239】【32†L599-L604】. Check logs for `zend_mm_heap corrupted`.
- **Patch:** Immediately update to PHP 8.2.35+ (or 8.3.36, 8.4.27, 8.5.12) which fix both bugs【37†L133-L142】.
- **Disable upload progress:** As a temporary mitigation, set `session.upload_progress.enabled=0` and restart Apache【41†L191-L198】. This removes the core callback the exploit relies on.
- **Harden limits:** Make `memory_limit = -1` (no PHP OOM) or enforce small `post_max_size` at the web server【41†L176-L184】. (This avoids the controlled OOM path.)
- **Other PHP settings:** The PHP advisory also suggests disabling fatal error backtraces (`fatal_error_backtraces=0`) on PHP 8.5 to block one RCE variant【41†L195-L201】.
- **Least privilege:** Run PHP as a low-privilege user, use a read-only docroot, disable dangerous functions, etc. (Stage 3 payloads usually require only shell execution as www-data.)

All this is for lab/research purposes. **Do not run these exploit tools against untrusted or third-party servers.** They are disruptive (large uploads, OOMs, worker crashes) and intended for **authorized security testing only**【32†L589-L598】【9†L19-L24】.

Sources: In writing this, we drew on the PHP official advisories【37†L133-L142】【41†L176-L184】, Wiz Research's technical blog【25†L245-L254】【25†L252-L259】, and the devploit/NCSC-NL php2shell repositories【18†L307-L314】【19†L265-L274】. These outline the vulnerability, lab setup, and exploit toolkit usage in detail. All commands and examples above are taken from those references or derived from running the devploit toolkit locally.
